11.01 Limitations of a file-based approach

Data integrity problems in a single file

Let's consider a simple scenario. A theatrical agency makes bookings for bands and is setting up a computerised system. Text files are to be used. One of these text files is to store data about individual band members. Each line of the file is to contain the following data for one band member:

- Name
- Contact details
- Banking details
- Band name
- Band agent name
- Band agent contact details

The intention is that this file could be used if the agency needed to contact the band member directly or through the band's agent. It could also be used after a gig when the band member has to be paid. Ignoring what would constitute contact details or banking details, we can look at a snapshot of some of the data that might be stored for the member's given name, the member's family name and the band name. The file might have a thousand or more lines of text. The following is a selection of some of the data that might be contained in various lines in the file:


```
Xiangfei    Jha         ComputerKidz
Mahesh      Ravuru      ITWizz
Dylan       Stoddart
Graham      Vandana     ITWizz
Vandana     Graham      ITWizz
Mahesh      Ravuru      ITWizz
Precious    Olsen       ComputerKidz
Precious    Olsen       ITWizz
```

It is clear that there are problems with this data. It would appear that when the data for Vandana Graham was first entered, her names were inserted in the wrong order. A later correct entry was made without deletion of the original incorrect data. This type of problem is not unique to a file-based system. There is no validation technique that could detect the original error. By contrast, validation should have led to the correction of the missing band name for Dylan Stoddart. The Precious Olsen data are examples of duplication of data and inconsistent data.

There is also possibly an error that is not evident from looking at the file contents. A band name could be entered here when that band doesn't exist.

The above discussion shows how a file-based approach can lead to data integrity problems in an individual file. The reason is the lack of in-built control when data is entered. The database approach can prevent such problems or, at least, minimise the chances of them happening.

The data privacy issue with a single file

A different problem is a lack of data privacy. The file above was designed so that the finance section could find banking details and the recruitment section could find contact details. The problem is that there cannot be any control of access to part of a file, so for example, staff in the recruitment section would be able to access the banking details of band members. Data privacy would be properly handled by a database system.

Data redundancy and possible inconsistency in multiple files

Mindful of this privacy problem, the agency decides to store data in different files for different departments of the organisation. Table 11.01 summarises the main data to be stored in each department's file.

Department	Data items in the section's file				
Contract	Member names		Band name	Gig details	
Finance	Member names	Bank details		Gig details	
Publicity			Band name	Gig details	
Recruitment	Member names		Band name		Agent details

Table 11.01 Data to be held in the department files

There is now data duplication across the files. This is commonly referred to as **data redundancy**. This does not mean that the data is no longer of use. Rather, it is a recognition that once data has been stored in one file there should be no need for it to be stored again in a different file. Unfortunately, some data redundancy cannot be avoided in file-based systems. This can lead to data inconsistency, either because of errors in the original entry or because of errors in subsequent editing. This is a different cause of data lacking integrity. One of the primary aims of the database approach is the elimination of data redundancy.



KEY TERM

Data redundancy: the same data stored more than once

Data dependency concerns

The above account has focused on the problems associated with storing the data in the files. We now need to consider the problems that might occur when programs access the files.

Traditionally a programmer wrote a program and at the same time defined the data files that the program would need. For the agency, each department would have its own programs that would access the department's data files. When a programmer creates a program for a department, the programmer has to know how the data is organised in these files, for example, that the fourth item on a line in the file is a band name. This is an example of 'data dependency'.

It is very likely that the files used by one department might have some data which is the same as the data in the files of other departments. However, in the scenario presented above there is no plan for file sharing.

A further issue is that the agency might decide that there is a need for a change in the data stored. For instance, they might see an increasing trend for bands to perform with additional session musicians. Their data will need to be entered into some files. This will require the existing files to be re-written. In turn, this will require the programs to be re-written so that the new files are read correctly. In a database scenario the existing programs could still be run even though additional data was added. The only programming change needed would be the writing of additional programs to use this additional data.

The other aspect of data dependency is that when file structures have been defined to suit specific programs, they may not be suited to supporting new applications. The agency might feel the need for an information system to analyse the success or otherwise of the gigs they have organised over a number of years. Extracting the data for this from the sort of file-based system described here would be a complex task that would take considerable time to complete.

11.02 The relational database

In the relational database model, each item of data is stored in a **relation** which is a special type of table. The strange choice of name comes from a mathematical theory. A relational database is a collection of relational tables.

When a table is created in a relational database it is first given a name and then the attributes are named. In a database design, a table would be given a name with the **attribute** names listed in brackets after the table name. For example, the design for a database for the theatrical agency might contain the table definitions shown in Figure 11.01.

Member(MemberID, MemberGivenName, MemberFamilyName, BandName, ...)

Band(BandName, AgentID, ...)

Figure 11.01 Two tables in a database design for the theatrical agency

A logical view of some data stored in these tables is given in Table 11.02 and Table 11.03. Each attribute is associated with one column in the table and is in effect a column header. The entries in the rows beneath this column header are attribute values.

MemberID	MemberGivenName	MemberFamilyName	BandName	...
0005	Xiangfei	Jha	ComputerKidz	...
0009	Mahesh	Ravuru	ITWizz	...
0001	Dylan	Stoddart	ComputerKidz	...
0025	Vandana	Graham	ITWizz	...

Table 11.02 Logical view of the Member table in a relational database

BandName	AgentID	...
ComputerKidz	01	...
ITWizz	07	...

Table 11.03 Logical view of the Band table in a relational database

This is described as a logical view because an underlying principle for a relational database is that there is no ordering defined for the attribute columns. At least one database product does allow a view of a table and its contents. However, this is not in keeping with the fundamental relational database concept that a query should be used to inspect the data in a table. Queries are discussed later in the chapter.

KEY TERMS

Relation: the special type of table which is used in a relational database

Attribute: a column in a relation that contains values

Tuple: a row in a relation storing data for one instance of the relation

A row in a relation should be referred to as a **tuple** but this formal name is not always used. Often a row is called a 'record' and the attribute values 'fields'. The tuple is the collection of data stored for one 'instance' of the relation. In Table 11.02, each tuple relates to one individual band member. A fundamental principle of a relational database is that a tuple is a set of 'atomic' values; each attribute has one value or no value.

The most important feature of the relational database concept is the **primary key**. A primary key may be a single attribute or a combination of attributes. Every table must have a primary key and each tuple in the table must have a value for the primary key and that value must be unique.

Once a table and its attributes have been defined, the next task is to choose the primary key. In some cases there may be more than one attribute for which unique values are guaranteed. In this case, each one is a **candidate key** and one will be selected as the primary key. A candidate key that is not selected as the primary key is then referred to as a **secondary key**. Often there is no candidate key and so a primary key has to be created. The design in Figure 11.01 illustrates this with the introduction of the attribute MemberID as the primary key for the Member table. Note that the primary key is underlined in the database design.

The primary key ensures integrity within the table. The DBMS will not allow an attempt to insert a value for a primary key when that value already exists. Therefore, each tuple automatically becomes unique. This is one of the features of the relational model that helps to ensure data integrity. The primary key also provides a unique reference to any attribute value that a query selects.

A database can contain stand-alone tables, but it is more usual for each table to have some relationship to another table. This relationship is implemented by using a **foreign key**.



KEY TERMS

Primary key: an attribute or a combination of attributes for which there is a value in each tuple and that value is unique

Candidate key: a key that could be chosen as the primary key

Secondary key: a candidate key that has not been chosen as the primary key

Foreign key: an attribute in one table that refers to the primary key in another table

Let's discuss the use of a foreign key using the database design shown in Figure 11.01. When the database is being created, the Band table is created first. BandName is chosen as the primary key because unique names for bands can be guaranteed. Then the Member table is created. MemberID is defined as the primary key and the attribute BandName is identified as a foreign key referencing the primary key in the Band table. Once this relationship between primary and foreign keys has been established, the DBMS will prevent any entry for BandName in the Member table being made if the corresponding value does not exist in the Band table. This provides **referential integrity** which is another reason why the relational database model helps to ensure data integrity.

Question 11.01

BandName is a primary key for the Band table. Does this mean that as a foreign key in the Member table it must have unique values? Explain your reasoning.



KEY TERM

Referential integrity: the use of a foreign key to ensure that a value can only be entered in one table when the same value already exists in the referenced table

11.03 Entity–relationship modelling

We can use a top-down method called stepwise refinement to break down the process of database design into simple steps (see also Chapter 12, Section 12.08). At each step more detail is added to the design. In database design this approach uses an entity-relationship (E–R) diagram. Typically, this can be created either by a database designer or a systems analyst working with the designer. We introduced the term 'relationship' earlier in connection with the use of a foreign key. An entity (strictly speaking an entity type) could be a thing, a type of person, an event, a transaction or an organisation. Most importantly, there must be a number of 'instances' of the entity. An entity is something that will become a table in a relational database.

WORKED EXAMPLE 11.01

Creating an entity–relationship diagram for the theatrical agency

Let's consider a scenario for the theatrical agency which will be sufficient to model a part of the final database they would need. The starting point for a top-down design is a statement of the requirement:

The agency needs a database to handle bookings for bands. Each band has a number of members. Each booking is for a venue. Each booking might be for one or more bands.

Step 1: Choose the entities

You look for the nouns. You ignore 'agency' because there is only the one. You choose Booking, Band, Member and Venue. For each of these there will be more than one instance. You are aware that each booking is for a gig at a venue but you ignore this because you think that the Booking entity will be sufficient to hold the required data about a gig.

Step 2: Identify the relationships

This requires experience, but the aim is not to define too many. You choose the following three:

- Booking with Venue
- Booking with Band
- Band with Member.

You ignore the fact that there will be, for example, a relationship between Member and Venue because you think that this will be handled through the other relationships that indirectly link them. You can now draw a preliminary E–R diagram as shown in Figure 11.02.



Figure 11.02 A preliminary entity–relationship diagram

Step 3: Decide the cardinalities of the relationships

Now comes the crucial stage of deciding on what are known as the 'cardinalities' of the relationships. At present we have a single line connecting each pair of entities. This line actually defines two relationships which might be described as the 'forward' one and the 'backward' one on the diagram as drawn. However, this only becomes apparent at the final stage of drawing the relationship. First, we have to choose one of the following descriptions for the cardinality of each relation:

- one-to-one or 1:1
- one-to-many or 1:M
- many-to-one or M:1
- many-to-many or M:M.

(Continued)